BOOMI REUSABLE FRAMEWORK · DATABASE ERRORS

Boomi DB Error Framework

Six shared components that any process can plug into its database steps. Errors are classified from the SQL error code, and only transient errors are retried, using Boomi's own back-off. Constraint and business failures go to a replayable error table. Fatal errors alert immediately and fail the execution. Every error is logged on a separate connection, so the log survives when the main pool is the thing that broke.

Start · Logic / control · Data shaping · Connector · Process call · Error handling · Stop / return

---

00 · COMPONENTS

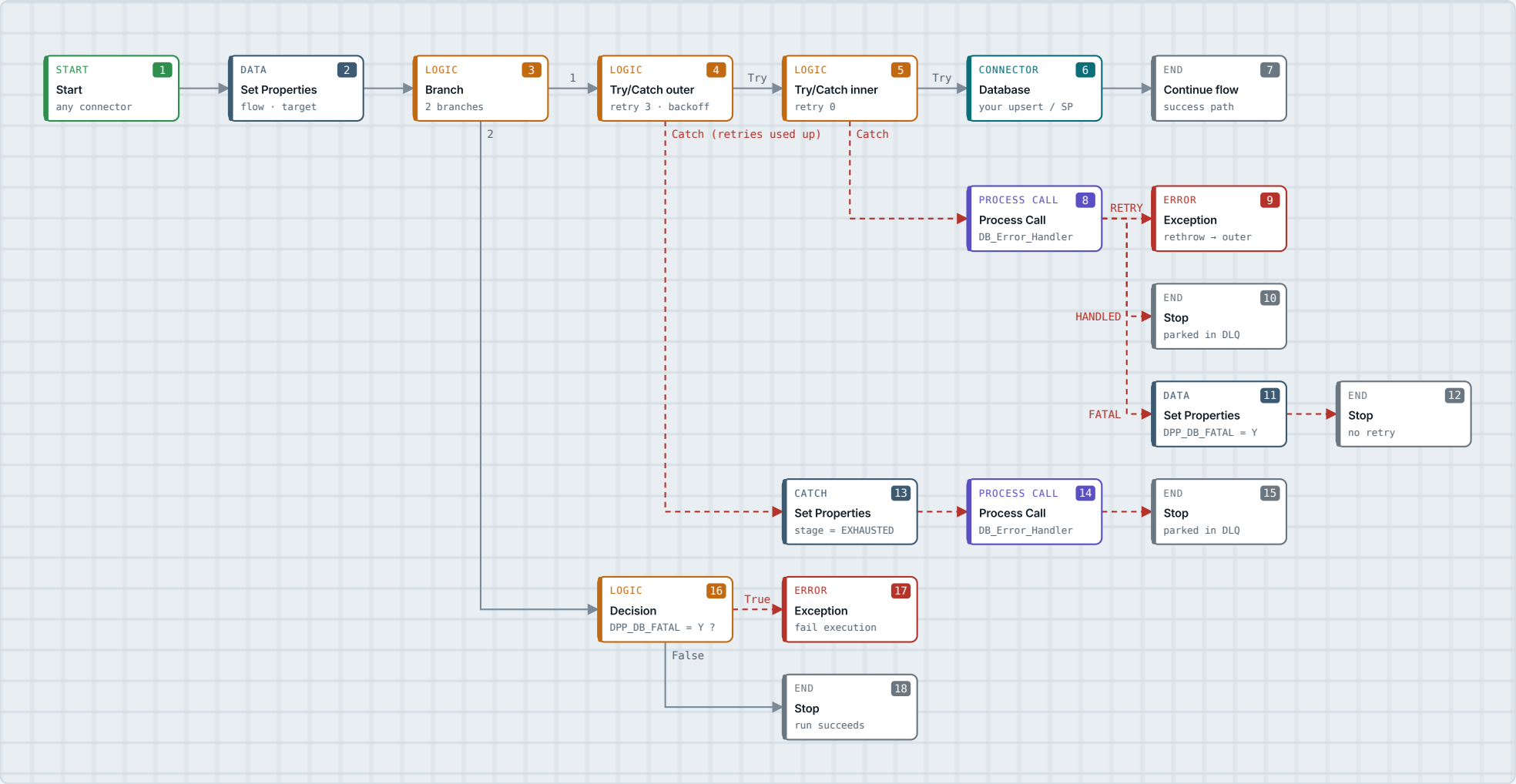
What lives in the shared folder

COMPONENT	TYPE	ROLE
SP_DB_Error_Handler	Process (sub process)	Captures context, classifies, logs, and returns one action: RETRY, HANDLED or FATAL
SP_DB_Error_Log	Process (sub process)	Writes one row to DB_ERROR_LOG with the failed payload. Never throws.
PP_DB_ERROR_RULES	Process Property component	Regex lists per class plus max payload size. Environment-extensible, so new codes need no redeploy.
CONN_DB_ERRLOG	Database connection	Separate pool for the log table. A dead business pool can't block error logging.
PR_DB_Error_Replay	Process (scheduled)	Re-sends rows ops marked REPLAY through a Process Route to the original write step
PR_<Flow>_Template	Process (copy me)	The calling pattern: nested Try/Catch + Branch, shown in section 01

01 · CALLING PATTERN

PR_<Flow>_Template

Outer Try/Catch: retry 3 | Inner Try/Catch: retry 0 | Branch 2 fails the run on FATAL



The trick is the two Try/Catch shapes. The inner one catches every DB error and asks the handler what to do. Only a RETRY answer rethrows, and that reaches the outer Try/Catch,

whose built-in retry runs the write again with increasing delays. Constraint, business and fatal errors end inside the inner catch, so they are never retried.

2 Set Properties — framework inputs

DATA

PROPERTY	VALUE	WHY
DPP_FLOW_NAME	`Static: "CUSTOMER_SYNC"`	Process Route key for replay
DPP_DB_TARGET	`Static: "ERP_DB"`	Which database failed
DPP_DB_OPERATION	`Static: "UPSERT_CUSTOMER"`	Which statement failed
DDP_BUSINESS_KEY	`Profile Element › customerId`	Lets ops find the record; per document
DPP_DB_FATAL	`Static: "N"`	Flipped by the FATAL path

3 Branch

CONTROL

PROPERTY	VALUE	WHY
Number of Branches	2	Branch 2 runs after branch 1 finishes and sees its DPPs

4 Try/Catch — outer (retry engine)

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger	`All Errors`	Catches the rethrow from shape 9
Retry Count	3	Boomi waits longer before each retry (roughly 10 s, 30 s, 60 s); max 5

5 Try/Catch — inner (classifier gate)

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger	`All Errors`	
Retry Count	`0`	Retry decisions belong to the handler, not to this shape

6 Database — your write

CONNECTOR

PROPERTY	VALUE	WHY
Action	`Send (or Get)`	Any operation; the framework doesn't care
SQL	`Upsert / MERGE, or a stored procedure`	Idempotent, so retries and replays are safe
Commit Option	`Commit By Profile`	Each document is its own unit; one bad row doesn't roll back 499 good ones

> Boomi has no transaction that spans several shapes. When two tables must change together, put both writes in one stored procedure.

8·14 Process Call — SP_DB_Error_Handler

EXEC

PROPERTY	VALUE	WHY
Wait / Abort if fails	`On / Off`	
Return paths	`RETRY · HANDLED · FATAL`	
Shape 14 input	`DDP_ERR_STAGE = EXHAUSTED (shape 13)`	Tells the handler retries are used up, so it parks the record instead of asking to retry

9 Exception — rethrow

ERROR

PROPERTY	VALUE	WHY
Stop a single document	On	Only the failed document is retried
Message	DB_TRANSIENT {1}: {2}	{1} = DDP_ERR_CODE, {2} = DDP_ERR_MSG

11 Set Properties — mark fatal

DATA

PROPERTY	VALUE	WHY
DPP_DB_FATAL	Static: "Y"	Read by branch 2

> FATAL ends in Stop here, inside the try scope. An Exception at this point would be caught by the outer Try/Catch and retried three times for nothing.

16·17 Decision + Exception — fail the run

CONTROL

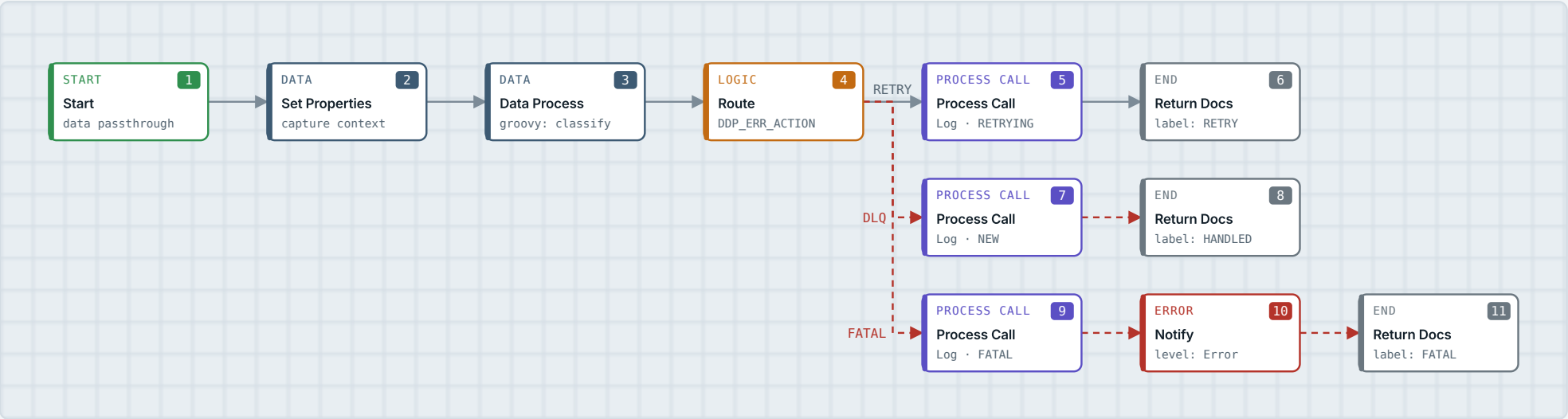
PROPERTY	VALUE	WHY
Decision	DPP_DB_FATAL Equal To "Y"	
Exception message	Fatal DB error on {1}; see DB_ERROR_LOG	Outside every Try/Catch, so the execution shows as Error and schedulers and alerts see it

---

02 · SUB PROCESS

SP_DB_Error_Handler

Start: Data Passthrough · Returns: RETRY · HANDLED · FATAL



2 Set Properties — capture context

DATA

PROPERTY	VALUE	WHY
DDP_ERR_MSG	Meta Information › Base › Try/Catch Message	The raw driver message
DDP_PROCESS_NAME	Execution Property › Process Name	
DDP_EXECUTION_ID	Execution Property › Execution Id	Links to Process Reporting
DDP_RUNTIME	Execution Property › Atom Name	Which node / environment
DDP_FLOW · DDP_DB_TARGET · DDP_DB_OP	Process Property › DPP_FLOW_NAME, DPP_DB_TARGET, DPP_DB_OPERATION	Copied to DDPs so they reach the log row and survive replay

3 Data Process — classify

DATA

PROPERTY	VALUE	WHY
Processing Step	Custom Scripting · Groovy 2.4	Sets DDP_ERR_CODE, DDP_ERR_CLASS, DDP_ERR_ACTION

```
import com.boomi.execution.ExecutionUtil
String P = "document.dynamic.userdefined."
// Optional overrides from PP_DB_ERROR_RULES (use the property KEY ids, not labels)
String PP = "<PP_DB_ERROR_RULES component id>"
def pp = { k, dflt -> (ExecutionUtil.getProcessProperty(PP, k) ?: dflt) }

def rules = [   // checked in this order; first match wins
  CONSTRAINT: pp("<key:CONSTRAINT>",
    /23505|23503|23502|23514|ORA-00001|ORA-0229[12]|ORA-01400|\b2627\b|\b2601\b|\b547\b|\b515\b|duplicate key|unique constraint|foreign key|cannot
be null/),
  FATAL: pp("<key:FATAL>",
    /28000|28P01|42P01|42703|42501|ORA-01017|ORA-00942|ORA-00904|ORA-0165\d|\b18456\b|\b208\b|\b207\b|login failed|password
authentication|permission denied|does not exist|invalid column/),
  TRANSIENT: pp("<key:TRANSIENT>",
    /08\d{3}|40001|40P01|57014|53300|ORA-00060|ORA-12170|ORA-125\d\d|ORA-0311[34]|\b1205\b|\b1222\b|\b-2\b|deadlock|timed? ?out|connection
(reset|refused|closed)|pool/)
]

for (int i = 0; i < dataContext.getDataCount(); i++) {
  InputStream is = dataContext.getStream(i)
  Properties p = dataContext.getProperties(i)
  String msg = p.getProperty(P + "DDP_ERR_MSG") ?: ""
  String cls = p.getProperty(P + "DDP_ERR_CLASS")          // caller may preset BUSINESS
  if (!cls) {
    cls = rules.find { k, rx -> msg =~ ("(?i)" + rx) }?.key ?: "UNKNOWN"
  }
  def m = msg =~ /ORA-\d{5}|(?i)sqlstate[:= ]*\w{5}|(?i)error code[:= ]*-?\d+/
  p.setProperty(P + "DDP_ERR_CODE", m.find() ? m.group() : "N/A")

  boolean exhausted = p.getProperty(P + "DDP_ERR_STAGE") == "EXHAUSTED"
  String action =
      cls == "TRANSIENT" ? (exhausted ? "DLQ" : "RETRY") :
      cls in ["CONSTRAINT", "BUSINESS"] ? "DLQ" :
      "FATAL"                                              // FATAL and UNKNOWN
  p.setProperty(P + "DDP_ERR_CLASS", cls)
  p.setProperty(P + "DDP_ERR_ACTION", action)
```

```
  dataContext.storeStream(is, p)
}
```

> UNKNOWN is treated as FATAL. A new error you haven't seen before should page someone once and get a rule, not be retried silently forever.

4 Route — action

CONTROL

PROPERTY	VALUE	WHY
Route by	`Document Property › DDP_ERR_ACTION`	
Keys	`RETRY · DLQ · FATAL`	Default → same as FATAL

5·7·9 Process Call — SP_DB_Error_Log

EXEC

PATH	DDP_LOG_STATUS	EFFECT
RETRY	`RETRYING`	Row for trend analysis; not replayable
DLQ	`NEW`	Replayable once ops fix the data or set REPLAY
FATAL	`FATAL`	Replayable after the fix (e.g. grant restored)

> Set DDP_LOG_STATUS with a Set Properties shape just before each call (left out of the diagram). The log sub process ends in Stop, so the original documents continue after the call.

10 Notify — fatal

ERROR

PROPERTY	VALUE	WHY
Level	`Error`	Subscribe in Platform alerts (email / webhook / ITSM)
Message	`[{1}] {2} on {3}: {4}`	class, code, DB target, message

6·8·11 Return Documents

END

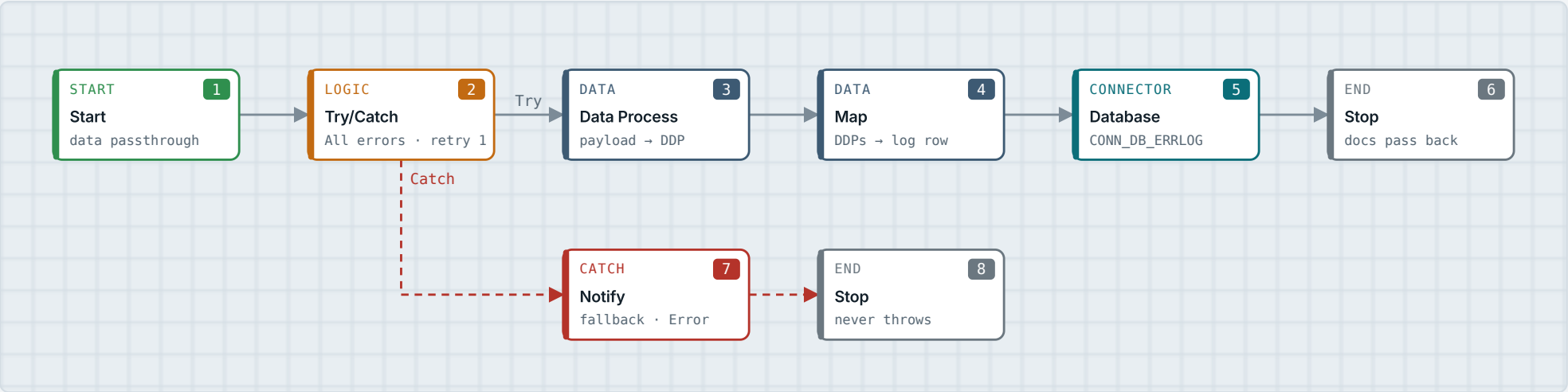
PROPERTY	VALUE	WHY
Labels	`RETRY · HANDLED · FATAL`	Become the caller's return paths

---

03 · SUB PROCESS

SP_DB_Error_Log

Start: Data Passthrough | Connection: CONN_DB_ERRLOG | Never throws



2 Try/Catch

CONTROL

PROPERTY	VALUE	WHY
Failure Trigger / Retry	`All Errors / 1`	One quick retry, then fall back to Notify. Logging must never break the caller.

3 Data Process — payload to DDP

DATA

PROPERTY	VALUE	WHY
Processing Step	`Custom Scripting · Groovy`	Copies the body into DDP_PAYLOAD, truncated to the PP max (e.g. 1 MB)

4 Map — context → DB_ERROR_LOG

DATA

PROPERTY	VALUE	WHY
Source profile	`Flat File with one dummy field`	Every destination field comes from functions
Destination profile	`Database Write · DB_ERROR_LOG insert`	
Functions	`Get Document Property for each DDP_*`	No logic in the map

5 Database — insert log row

CONNECTOR

PROPERTY	VALUE	WHY
Connection	`CONN_DB_ERRLOG`	Own pool, so it works when the business pool is exhausted
Action / Commit	`Send / Commit By Profile`	Each row commits on its own

7 Notify — fallback

ERROR

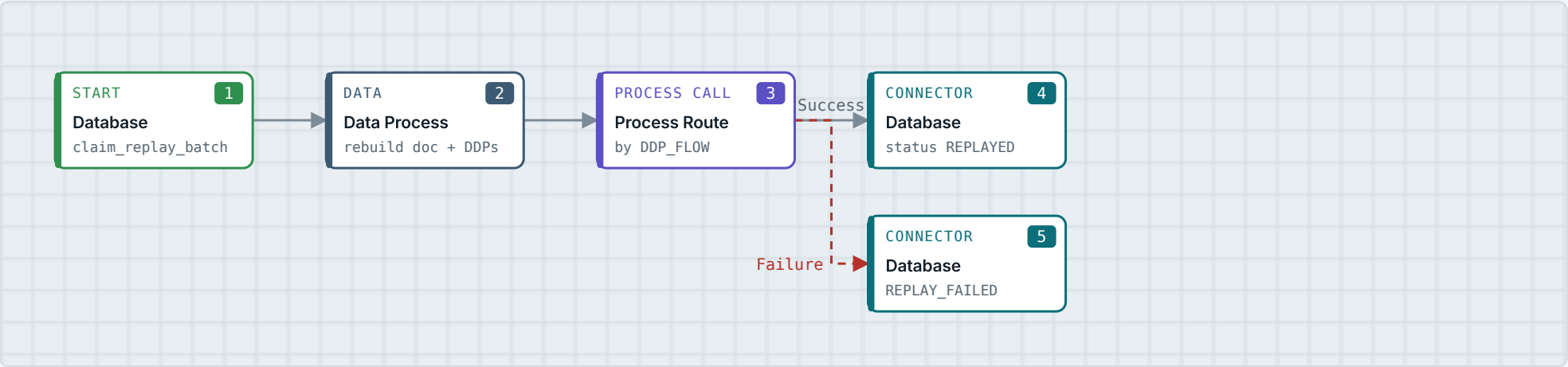
PROPERTY	VALUE	WHY
Level / Message	`Error / LOG WRITE FAILED · {1} {2} key={3}: {4}`	The error still reaches Process Reporting even if the log DB is down

---

04 · PROCESS

PR_DB_Error_Replay

Schedule: every 10 min | Allow Simultaneous Executions: Off | Process Route by flow



1 Start — Database get

START

PROPERTY	VALUE	WHY
Connection	CONN_DB_ERRLOG	
SQL	SELECT * FROM claim_replay_batch(100)	Atomically flips REPLAY → REPLAYING, so two runs can't take the same row

2 Data Process — rebuild document

DATA

PROPERTY	VALUE	WHY
Processing Step	Custom Scripting	Body = payload column; restores DDP_FLOW, DDP_BUSINESS_KEY and DDP_ERROR_ID

3 Process Route

EXEC

PROPERTY	VALUE	WHY
Route by	`Document Property › DDP_FLOW`	One route key per flow, e.g. CUSTOMER_SYNC → SP_Customer_DB_Write
Return paths	`Success · Failure`	Defined on the Process Route component

> This is why the template's write step should live in its own Data Passthrough sub process (SP_<Flow>_DB_Write). The live flow and the replay then call exactly the same code.

4·5 Database — close the row

CONNECTOR

PATH	SQL	WHY
Success	`UPDATE … SET status='REPLAYED', resolved_at=now() WHERE error_id=?`	
Failure	`UPDATE … SET status='REPLAY_FAILED', replay_count=replay_count+1`	A new error row is also written by the handler inside the replayed flow

---

05 · CLASSIFICATION

Default code map

CLASS	POSTGRESQL SQLSTATE	ORACLE	SQL SERVER	ACTION
Transient	`08xxx, 40001, 40P01, 57014, 53300`	ORA-00060, ORA-12170, ORA-125xx, ORA-03113/4	1205, 1222, -2 (timeout)	RETRY ×3 → DLQ
Constraint	`23505, 23503, 23502, 23514`	ORA-00001, ORA-02291/2, ORA-01400	2627, 2601, 547, 515	DLQ, no retry

Business	Raised by the flow: preset DDP_ERR_CLASS = BUSINESS (e.g. 0 rows updated, parent missing) before calling the handler			DLQ, no retry
Fatal	28000, 28P01, 42P01, 42703, 42501	ORA-01017, ORA-00942, ORA-00904, ORA-0165x	18456, 208, 207	Alert + fail run
Unknown	No pattern matched			Same as Fatal

---

06 · SHARED STATE

Error table and replay claim (PostgreSQL)

```
create table db_error_log (
  error_id       bigserial primary key,
  created_at     timestamptz not null default now(),
  runtime        varchar(100),
  process_name   varchar(200) not null,
  execution_id   varchar(100) not null,
  flow_name      varchar(80)  not null,          -- Process Route key
  db_target      varchar(80),
  db_operation   varchar(120),
  business_key   varchar(200),
  err_class      varchar(12)  not null,          -- TRANSIENT | CONSTRAINT | BUSINESS | FATAL | UNKNOWN
  err_code       varchar(20),
  err_message    text,
  payload        text,                           -- failed document, truncated to PP max
  status         varchar(14)  not null,          -- RETRYING | NEW | FATAL | REPLAY | REPLAYING | REPLAYED | REPLAY_FAILED | IGNORED
  replay_count   int not null default 0,
  resolved_at    timestamptz,
  resolved_by    varchar(100)
);
create index ix_dberr_status on db_error_log(status, created_at);
create index ix_dberr_key    on db_error_log(flow_name, business_key);

-- Ops: after fixing the data,  UPDATE db_error_log SET status='REPLAY' WHERE error_id IN (...);

create or replace function claim_replay_batch(p_limit int)
returns setof db_error_log as $$
  update db_error_log e set status = 'REPLAYING'
```

```
   where e.error_id in (select error_id from db_error_log
                        where status = 'REPLAY'
                        order by error_id
                        limit p_limit
                        for update skip locked)
  returning e.*;
$$ language sql;

-- Housekeeping: keep RETRYING rows 30 days, resolved rows 180 days
delete from db_error_log where status = 'RETRYING' and created_at < now() - interval '30 days';
delete from db_error_log where status in ('REPLAYED','IGNORED') and resolved_at < now() - interval '180 days';
```

---

07 · CONNECTIONS

Pool settings on every DB connection

SETTING	BUSINESS CONNECTION	CONN_DB_ERRLOG
Enable Pooling	On	On
Maximum Connections	Sized to Flow Control threads × nodes (e.g. 20)	5
When exhausted	Wait, max 30 s	Wait, max 5 s
Test on borrow + validation query	On · SELECT 1	On · SELECT 1
Max idle time	Below the DB's idle-kill timeout	Same
Credentials	Environment extension, never in the component	Same; separate DB user with INSERT/UPDATE on db_error_log only

> Field labels on the Database connection's pooling tab differ slightly between the legacy and V2 connectors; match by meaning.

- Alert when `count(*) where status in ('NEW','FATAL')` is above 0 for P1 flows, and on RETRYING rows per hour as an early warning for deadlocks and pool pressure.
- Add a new vendor code by editing the PP_DB_ERROR_RULES environment extension. No redeploy is needed.
- Your Try/Catch catch path should read `Meta Information › Base › Try/Catch Message`. That is the property Boomi exposes for the error text.